Unit 05

Bash Scripting Basics

Module 1 — Technical Foundations · ~5 class periods

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Why automate?

- A script is a text file of commands the computer runs in order — like a recipe.
- Automation gives you **consistency, speed, fewer mistakes, repeatability**.
- In security: scan 254 hosts in seconds; reproduce a result for a report exactly.

> A script is just the commands you already learned in Unit 04 — typed into a file.

Learning objectives (1 of 2)

By the end of this unit you can:

- **Explain** why automation matters and give two examples.
- **Write** a runnable script with a shebang, `chmod +x` it, run it with `./script.sh`.
- **Create and use** variables, **read** input, and **capture** command output with `$(...)`.
- **Use** positional arguments (`$1`, `$@`, `$#`).

Learning objectives (2 of 2)

- **Write** `if` conditionals using `test` / `[ ]`.
- **Build** `for` and `while` loops.
- **Define and call** a function; explain the exit code `$?`.
- **Combine** tools with pipes (`|`).
- **Build** a working **ping sweep** that reports live hosts on the **isolated lab subnet**.
- **Apply** the rule: network scripts run **only** against the lab subnet.

Key vocabulary (1 of 2)

Term	Meaning
Script	A text file of commands run in order.
Bash	The shell + language we script in this unit.
Shebang	First line `#!/bin/bash` — says which program runs the file.
Executable	A file allowed to run as a program (set with `chmod +x`).
Variable	A named box holding a value.
Command substitution	Capturing a command's output: `$(command)`.
Argument	Info you pass to a script when you run it.

Key vocabulary (2 of 2)

Term	Meaning
Positional parameter	`$1` first arg, `$2` second, `$@` all, `$#` how many.
Conditional	Code that runs only `if` a test is true.
Test / `[ ]`	How Bash checks if something is true.
Loop	Repeats code: `for` a set number, `while` a condition holds.
Function	A named, reusable block of code.
Exit code	Number a command returns: `0` = success. Stored in `$?`.
Ping sweep	Pinging a range of addresses to find live hosts.
Subnet (/24)	A block of 256 IPs sharing the first three numbers.
Host-only network	Virtual net connecting VMs — **not** the internet/school.

⚖️ A script does not change the law

Automation lets you break the rules *faster*.

A ping sweep is active scanning. The **only** legal target this week is the isolated host-only lab subnet your instructor assigned.

Ethics + scope check

- Scanning a network you don't own / lack permission for can violate the **CFAA** and state law.
- **Before any network command**, confirm your VM is on the host-only adapter:

```
ip addr        # expect a 192.168.56.x (or your lab) address
```

- If you don't see the lab subnet, **stop and ask** — you may be on the wrong adapter.

Discussion: A one-line loop pings all 254 addresses in seconds. Why does that speed make the authorization rule *more* important, not less?

Day 1

Why automate, and your first script

Anatomy of a script

```
#!/bin/bash
# hello.sh — my first script
echo "Hello from Bash!"
```

- `#!/bin/bash` — the **shebang**: tells the system to run this with Bash.
- `#` starts a **comment** (ignored by Bash, read by humans).
- `echo` prints a line of text.

Making it run

```
chmod +x hello.sh      # turn ON the execute permission
./hello.sh             # run it
```

- `chmod +x` makes the file **executable** (remember rwx from Unit 04).
- The `./` says "run the file **here**", not a command on the system path.
- Forget `chmod` → `Permission denied`. Forget `./` → `command not found`.

> Two steps turn text into a program: `chmod +x` then `./`.

Day 1 lab — recon banner (start)

```
nano banner.sh
```

```
#!/bin/bash
# banner.sh — prints a simple recon banner
echo "==== Recon Banner ===="
echo "Operator: <put your name here>"
```

Then:

```
chmod +x banner.sh
./banner.sh
```

Day 2

Variables, input & command substitution

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Variables

```
name="Kali"           # NO spaces around the =
echo "$name"           # use it with a $
echo "Hi ${name}!"     # braces when next to other text
```

⚠️ The #1 beginner bug: `name = "Kali"` **fails**. It must be `name="Kali"` — no spaces around `=`.

- Always quote variables: `"$name"` avoids surprises with spaces.

Reading keyboard input

```
read -p "What's your name? " name
echo "Hello, $name"
```

- `read` pauses and stores what the user types into a variable.
- `-p` shows a prompt on the same line.

Command substitution: `$(...)`

Capture a command's **output** into a variable.

```
today=$(date)             # store the date command's output
echo "Date: $today"

myip=$(hostname -I)       # store this machine's IP
echo "This machine: $myip"
```

- `$(command)` runs the command and hands back its text.

Day 2 lab — extend the banner

```
#!/bin/bash
# banner.sh
echo "==== Recon Banner ===="
echo "Operator: Jordan Lee"
today=$(date)
echo "Date: $today"
myip=$(hostname -I)
echo "This machine: $myip"
read -p "What target are you working on today? " target
echo "Target for this session: $target"
```


Type a **lab** address only. That completes **Part 1**.

Exit ticket: Write the line that stores today's date in a variable named `today`.

Day 3

Arguments and conditionals

Positional arguments

When you run `./pingone.sh 192.168.56.1`:

Variable	Holds	Value
`$1`	first argument	`192.168.56.1`
`$2`	second argument	(none here)
`$@`	all arguments	`192.168.56.1`
`$#`	how many arguments	`1`

Arguments make a script **flexible**: one script, any (lab) target — no editing.

Conditionals: `if`

```
if [ $# -eq 0 ]; then
    echo "Usage: ./pingone.sh <ip>"
    exit 1
fi
```

- `[ ... ]` is the **test**. `if` / `then` /.../ `fi` wraps the decision.
- `-eq` = equal, `-lt` = less than, `-gt` = greater than (for numbers).
- `exit 1` stops the script with a non-zero code (= "problem").

Exit codes and `$?`

- Every command returns an **exit code** when it finishes.
- `0` **= success**, anything else = a problem.
- The last command's code is stored in `$?`.

```
ping -c 1 192.168.56.1 > /dev/null 2>&1
echo $?          # 0 if the host replied, non-zero if not
```

`ping` returns `0` when a host answers. That's how we'll detect "UP".

Day 3 lab — ping one host (start)

```
#!/bin/bash
# pingone.sh — ping one lab host given as $1
# SAFETY: lab subnet only.

if [ $# -eq 0 ]; then
    echo "Usage: ./pingone.sh <ip>"
    exit 1
fi

target=$1
echo "Pinging $target ..."
```


Exit ticket: After a command runs, where is its exit code stored, and what number

Day 3 lab — check the result

```
ping -c 1 -W 1 "$target" > /dev/null 2>&1

if [ $? -eq 0 ]; then
    echo "$target is UP"
else
    echo "$target is DOWN"
fi
```

- `-c 1` sends **one** ping; `-W 1` waits **at most 1 second**.
- `> /dev/null 2>&1` throws away ping's own output.
- `$?` must be read **immediately** after `ping`, before anything overwrites it.

Run it (lab only)

```
chmod +x pingone.sh
./pingone.sh 192.168.56.1      # a known-live lab address
./pingone.sh 192.168.56.250    # a known-down lab address
```

Expected: first prints `UP`, second `DOWN` after ~1 second.

> That's **Part 2 complete** — stopping here still earns full Part 2 credit.

Day 4

Loops, functions, and pipes

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

`for` loops

```
for i in 1 2 3; do
    echo "Number $i"
done

for host in $(seq 1 254); do
    echo "192.168.56.$host"
done
```

- `seq 1 254` produces the numbers 1 through 254.
- The loop runs the body once per value.

`while` loops

```
count=1
while [ $count -le 5 ]; do
    echo "Count is $count"
    count=$((count + 1))
done
```

- `for` repeats a set number of times; `while` repeats **as long as** a condition holds.
- `$((...))` does arithmetic.

Functions

```
check_host() {
    ping -c 1 -W 1 "$1" > /dev/null 2>&1
    if [ $? -eq 0 ]; then
        echo "$1 is UP"
    fi
}

check_host 192.168.56.1     # call it by name
```

- Define once, call by name. `$1` **inside** the function is the argument passed **to the function**.

Pipes recap

```
ping -c 1 192.168.56.1 | grep "bytes from"
```

- The pipe `|` sends one command's output into the next.
- Everything you learned in Unit 04 still applies inside scripts.

Day 4 lab — build the sweep (start)

```
#!/bin/bash
# sweep.sh — ping sweep of a lab /24, reports live hosts
# SAFETY: ISOLATED LAB SUBNET ONLY.

subnet="192.168.56"     # change only to your assigned lab subnet

check_host() {
    ping -c 1 -W 1 "$1" > /dev/null 2>&1
    if [ $? -eq 0 ]; then
        echo "$1 is UP"
    fi
}
```

Day 4 lab — loop the /24

```
echo "Sweeping $subnet.0/24 (lab only) ..."
for host in $(seq 1 254); do
    check_host "$subnet.$host"
done
echo "Sweep complete."
```

- The loop builds each full address (`192.168.56.7`, …) and checks it.
- Function + loop + conditional + exit code = a working tool.

Day 5

Finish, test & document the ping sweep

The complete `sweep.sh`

```
#!/bin/bash
# sweep.sh — ping sweep of a lab /24, reports live hosts
# SAFETY: ISOLATED LAB SUBNET ONLY. Do not point this anywhere else.

subnet="192.168.56"     # change only to your assigned lab subnet

check_host() {
    ping -c 1 -W 1 "$1" > /dev/null 2>&1
    if [ $? -eq 0 ]; then
        echo "$1 is UP"
    fi
}
```

Run it and check results

```
chmod +x sweep.sh
./sweep.sh
```

- Lists only the **UP** hosts, then `Sweep complete.`
- Cross-check the live list against your instructor's **lab inventory**.

Common bugs: space around `=`, inverted exit-code logic, wrong subnet, missing `-W 1` (sweep hangs on dead hosts and looks frozen).

Stretch goals

- Take the subnet as an argument: `./sweep.sh 192.168.56`.
- Count and print the total number of live hosts.
- Write the live list to a file with `>` and timestamp it with `$(date)`.
- Background each ping with `&` + `wait` for speed:

```
for host in $(seq 1 254); do
    check_host "$subnet.$host" &
done
wait
```

- Reject input that isn't a valid IP.

Lab deliverables

- Safety reminder in your own words + your assigned lab subnet.
- The three scripts: `banner.sh`, `pingone.sh`, `sweep.sh` (pasted text).
- A screenshot of each running.
- The **list of live hosts**, cross-checked against the lab inventory.
- A **line-by-line annotation** of `sweep.sh`.
- Reflection: one task you could automate + one place running it would be **illegal**.

Recap — what you can now do

- Write/run scripts: shebang, `chmod +x`, `./script.sh`.
- Variables, `read` input, command substitution `$(...)`.
- Arguments `$1` / `$@` / `$#`; conditionals with `[ ]`; exit codes `$?`.
- `for` and `while` loops; functions; pipes.
- Built a real **ping sweep** — for the **lab subnet only**.

Quiz preview (assessment)

- What does the shebang `#!/bin/bash` do?
- Which line correctly makes a variable? (`name="Kali"`)
- Where is the exit code stored and what means success? (`$?`, `0`)
- Spot the bug: `if [ $? -eq 1 ]` for "UP" — why is it backwards?
- Write a 5–8 line script that pings `$1` once and prints UP/DOWN.

Discussion / exit ticket

"Once it's a script, ping sweeping any network is fine — it's just automation."

Correct that — in 2–3 sentences using the words *authorization* and *scope*.

Name one task you could now automate, and one place it would be **illegal** to run it.

Next up

Unit 06: Python for Security — same ideas, new language, and a TCP port scanner.

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP
github.com/ajm4n · linkedin.com/in/aj-hammond

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP